

STEM BUILDERS



Functions

5. Introducing Functions

One of the core principles of any programming language is, "Don't Repeat Yourself". If you have an action that should occur many times, you can define that action once and then call that code whenever you need to carry out that action.

We are already repeating ourselves in our code, so this is a good time to introduce simple functions. Functions mean less work for us as programmers, and effective use of functions results in code that is less error-prone.

What are functions?

Functions are a set of actions that we group together, and give a name to. You have already used a number of functions from the core Python language, such as `string.title()` and `list.sort()`. We can define our own functions, which allow us to "teach" Python new behavior.

General Syntax

A general function looks something like this:

```
# Let's define a function.
def function_name(argument_1, argument_2):
    # Do whatever we want this function to do,
    # using argument_1 and argument_2
# Use function_name to call the function.
function_name(value_1, value_2)
```

This code will not run, but it shows how functions are used in general.

- **Defining a function**
 - Give the keyword `def`, which tells Python that you are about to *define* a function.
 - Give your function a name. A variable name tells you what kind of value the variable contains; a function name should tell you what the function does.
 - Give names for each value the function needs in order to do its work.
 - These are basically variable names, but they are only used in the function.
 - They can be different names than what you use in the rest of your program.
 - These are called the function's *arguments*.
 - Make sure the function definition line ends with a colon.
 - Inside the function, write whatever code you need to make the function do its work.

- **Using your function**
 - To *call* your function, write its name followed by parentheses.
 - Inside the parentheses, give the values you want the function to work with.
 - These can be variables such as `current_name` and `current_age`, or they can be actual values such as `'eric'` and `5`.

Basic Examples

For a simple first example, we will look at a program that compliments people. Let's look at the example, and then try to understand the code. First we will look at a version of this program as we would have written it earlier, with no functions.

```
print("You are doing good work, Adriana!")
print("Thank you very much for your efforts on this project.")

print("\nYou are doing good work, Billy!")
print("Thank you very much for your efforts on this project.")

print("\nYou are doing good work, Caroline!")
print("Thank you very much for your efforts on this project.")
```

Output:

```
You are doing good work, Adriana!
Thank you very much for your efforts on this project.
You are doing good work, Billy!
Thank you very much for your efforts on this project.
You are doing good work, Caroline!
Thank you very much for your efforts on this project.
```

Functions take repeated code, put it in one place, and then you call that code when you want to use it. Here's what the same program looks like with a function.

```
def thank_you(name):
    # This function prints a two-line personalized thank you message.
    print("\nYou are doing good work, %s!" % name)
    print("Thank you very much for your efforts on this project.")

thank_you('Adriana')
thank_you('Billy')
thank_you('Caroline')
```

Output:

```
You are doing good work, Adriana!
Thank you very much for your efforts on this project.
You are doing good work, Billy!
```

Thank you very much for your efforts on this project.
You are doing good work, Caroline!
Thank you very much for your efforts on this project.

In our original code, each pair of print statements was run three times, and the only difference was the name of the person being thanked. When you see repetition like this, you can usually make your program more efficient by defining a function.

The keyword `def` tells Python that we are about to define a function. We give our function a name, `thank_you()` in this case. A variable's name should tell us what kind of information it holds; a function's name should tell us what the variable does. We then put parentheses. Inside these parentheses we create variable names for any variable the function will need to be given in order to do its job. In this case the function will need a name to include in the thank you message. The variable `name` will hold the value that is passed into the function `thank_you()`.

To use a function we give the function's name, and then put any values the function needs in order to do its work. In this case we call the function three times, each time passing it a different name.

A common error

A function must be defined before you use it in your program. For example, putting the function at the end of the program would not work.

```
thank_you('Adriana')
thank_you('Billy')
thank_you('Caroline')
```

```
def thank_you(name):
    # This function prints a two-line personalized thank you message.
    print("\nYou are doing good work, %s!" % name)
    print("Thank you very much for your efforts on this project.")
```

OUTPUT:

```
-----
NameError                                Traceback (most recent call last)
<ipython-input-1-a1b6b8373f44> in <module>()
----> 1 thank_you('Adriana')
      2 thank_you('Billy')
      3 thank_you('Caroline')
      4
      5 def thank_you(name):
NameError: name 'thank_you' is not defined
```

On the first line we ask Python to run the function `thank_you()`, but Python does not yet know how to do this function. We define our functions at the beginning of our programs, and then we can use them when we need to.

A second example

When we introduced the different methods for sorting a list, our code got very repetitive. It takes two lines of code to print a list using a for loop, so these two lines are repeated whenever you want to print out the contents of a list. This is the perfect opportunity to use a function, so let's see how the code looks with a function.

First, let's see the code we had without a function:

```
students = ['bernice', 'aaron', 'cody']
# Put students in alphabetical order.
students.sort()

# Display the list in its current order.
print("Our students are currently in alphabetical order.")
for student in students:
    print(student.title())

# Put students in reverse alphabetical order.
students.sort(reverse=True)
# Display the list in its current order.
print("\nOur students are now in reverse alphabetical order.")
for student in students:
    print(student.title())
```

Output:

Our students are currently in alphabetical order.

Aaron
Bernice
Cody

Our students are now in reverse alphabetical order.

Cody
Bernice
Aaron

Here's what the same code looks like, using a function to print out the list:

```
def show_students(students, message):  
    # Print out a message, and then the list of students  
    print(message)  
    for student in students:  
        print(student.title())  
students = ['bernice', 'aaron', 'cody']  
# Put students in alphabetical order.  
students.sort()  
show_students(students, "Our students are currently in alphabetical order.")  
  
#Put students in reverse alphabetical order.  
students.sort(reverse=True)  
show_students(students, "\nOur students are now in reverse alphabetical order.")
```

Output:

Our students are currently in alphabetical order.

Aaron

Bernice

Cody

Our students are now in reverse alphabetical order.

Cody

Bernice

Aaron

This is much cleaner code. We have an action we want to take, which is to show the students in our list along with a message. We give this action a name, `show_students()`.

This function needs two pieces of information to do its work, the list of students and a message to display. Inside the function, the code for printing the message and looping through the list is exactly as it was in the non-function code.

Now the rest of our program is cleaner, because it gets to focus on the things we are changing in the list, rather than having code for printing the list. We define the list, then we sort it and call our function to print the list. We sort it again, and then call the printing function a second time, with a different message. This is much more readable code.

Advantages of using functions

You might be able to see some advantages of using functions, through this example:

- We write a set of instructions once. We save some work in this simple example, and we save even more work in larger programs.
- When our function works, we don't have to worry about that code anymore. Every time you repeat code in your program, you introduce an opportunity to make a mistake. Writing a function means there is one place to fix mistakes, and when those bugs are fixed, we can be confident that this function will continue to work correctly.
- We can modify our function's behavior, and that change takes effect every time the function is called. This is much better than deciding we need some new behavior, and then having to change code in many different places in our program.

For a quick example, let's say we decide our printed output would look better with some form of a bulleted list. Without functions, we'd have to change each print statement. With a function, we change just the print statement in the function:

```
def show_students(students, message):  
    # Print out a message, and then the list of students  
    print(message)  
    for student in students:  
        print("- " + student.title())  
students = ['bernice', 'aaron', 'cody']  
# Put students in alphabetical order.  
students.sort()  
show_students(students, "Our students are currently in alphabetical order.")  
#Put students in reverse alphabetical order.  
students.sort(reverse=True)  
show_students(students, "\nOur students are now in reverse alphabetical order.")
```

Output:

Our students are currently in alphabetical order.

- Aaron
- Bernice
- Cody

Our students are now in reverse alphabetical order.

- Cody
- Bernice
- Aaron

You can think of functions as a way to "teach" Python some new behavior. In this case, we taught Python how to create a list of students using hyphens; now we can tell Python to do this with our students whenever we want to.

Returning a Value

Each function you create can return a value. This can be in addition to the primary work the function does, or it can be the function's main job. The following function takes in a number, and returns the corresponding word for that number:

```
def get_number_word(number):  
    # Takes in a numerical value, and returns  
    # the word corresponding to that number.  
    if number == 1:  
        return 'one'  
    elif number == 2:  
        return 'two'  
    elif number == 3:  
        return 'three'  
    # ...  
# Let's try out our function.  
for current_number in range(0,4):  
    number_word = get_number_word(current_number)  
    print(current_number, number_word)
```

Output:

```
0 None  
1 one  
2 two  
3 three
```

It's helpful sometimes to see programs that don't quite work as they are supposed to, and then see how those programs can be improved. In this case, there are no Python errors; all of the code has proper Python syntax. But there is a logical error, in the first line of the output.

We want to either not include 0 in the range we send to the function, or have the function return something other than None when it receives a value that it doesn't know. Let's teach our function the word 'zero', but let's also add an else clause that returns a more informative message for numbers that are not in the if-chain.

```
def get_number_word(number):  
    # Takes in a numerical value, and returns  
    # the word corresponding to that number.  
    if number == 0:  
        return 'zero'  
    elif number == 1:  
        return 'one'  
    elif number == 2:  
        return 'two'  
    elif number == 3:  
        return 'three'
```



```
    else:
        return "I'm sorry, I don't know that number."

# Let's try out our function.
for current_number in range(0,6):
    number_word = get_number_word(current_number)
    print(current_number, number_word)
```

Output:

```
0 zero
1 one
2 two
3 three
4 I'm sorry, I don't know that number.
5 I'm sorry, I don't know that number.
```

If you use a return statement in one of your functions, keep in mind that the function stops executing as soon as it hits a return statement. For example, we can add a line to the `get_number_word()` function that will never execute, because it comes after the function has returned a value:

```
def get_number_word(number):
    # Takes in a numerical value, and returns
    # the word corresponding to that number.
    if number == 0:
        return 'zero'
    elif number == 1:
        return 'one'
    elif number == 2:
        return 'two'
    elif number == 3:
        return 'three'
    else:
        return "I'm sorry, I don't know that number."
    # This line will never execute, because the function has already
    # returned a value and stopped executing.
    print("This message will never be printed.")

# Let's try out our function.
for current_number in range(0,6):
    number_word = get_number_word(current_number)
    print(current_number, number_word)
```

Output:

```
0 zero
1 one
2 two
3 three
4 I'm sorry, I don't know that number.
5 I'm sorry, I don't know that number.
```

More Later

There is much more to learn about functions, but we will get to those details later. For now, feel free to use functions whenever you find yourself writing the same code several times in a program. Some of the things you will learn when we focus on functions:

- How to give the arguments in your function default values.
- How to let your functions accept different numbers of arguments.

Exercises

Max of Two/Three

- Write a Python function to find the Max of two numbers.
- **Bonus:** Write a Python function to find the Max of three numbers. 

Factorial

- Write a Python function to calculate the factorial of a number (a non-negative integer).
- The function accepts the number as an argument.

Range

- Write a Python function to check whether a number is in a given range.



Motivate to Innovate...



**STEM
BUILDERS**
Learning Center

MATH | ROBOTICS | TECHNOLOGY | EVENTS | CAMPS



contactus@stembuilders.com



www.stembuilders.com